EXPERIMENT NO: 8

Student Name: Dhruv Singh
UID: 23BCS11146
Branch: BE-CSE
Sec on/Group: KRG-2/A
Semester: 6th
Date of Performance: 24/03/2026
Subject Name: System Design
Subject Code: 23CSH-314

Aim

To design a scalable stock trading pla orm similar to Zerodha/Upstox that enables real- me market data streaming, order execu on, por olio tracking, and high-frequency trading with strong consistency and low latency.

Objec ves

- Understand the architecture of real- me stock trading systems.
- Iden fy func onal requirements such as trading, por olio tracking, and watchlists.
- Analyze non-func onal requirements including latency, scalability, and consistency.
- Apply CAP theorem trade-offs in financial systems.
- Design REST and WebSocket APIs for trading workflows.

Procedure

1. Studied real-world trading pla orms like Zerodha and Upstox.
2. Iden fied core en es such as Users, Stocks, Orders, Trades, and Por olio.
3. Analyzed real- me stock updates using WebSockets.
4. Collected func onal and non-func onal requirements.
5. Designed APIs for trading, funds management, and por olio tracking.
6. Evaluated consistency requirements for financial transac ons.
7. Analyzed low-latency system design for order execu on.

Func onal Requirements

1. Users should be able to register, login, and complete KYC verifica on.
2. Users should be able to view real- me stock prices and historical data.
3. Users should be able to place, modify, and cancel buy/sell orders.

4. System should support limit and market orders with accurate status tracking.
5. Users should be able to maintain a watchlist with real- me updates.
6. System should provide a dashboard for holdings, trade history, and P&L.
7. Users should be able to deposit and withdraw funds.

Core En es of the System

- User
- Stock
- Order
- Trade
- Por olio
- Watchlist

API Design

1. User API

A. Signup: POST /auth/signup

B. KYC Verifica on: POST /auth/verify-KYC

C. Login: POST /auth/login

2. Market Data API

A. Live Stock Feed (WebSocket): WS /api/v1/stocks

B. Stock Details: GET /api/v1/stocks/{symbol}

C. Historical Data: GET /api/v1/stocks/{symbol}/history

3. Funds API

A. Deposit Funds: POST /api/v1/funds/deposit

B. Withdraw Funds: POST /api/v1/funds/withdraw

C. Transac on History: GET /api/v1/funds/history

4. Trading API

A. Place Order: POST /api/v1/orders

B. Get Orders: GET /api/v1/users/{user_id}/orders
C. Order Status: GET /api/v1/orders/{order_id}

D. Cancel Order: DELETE /api/v1/orders/{order_id}

5. Por olio API

A. Por olio Overview: GET /api/v1/por olio B.

Holdings: GET /api/v1/por olio/holdings

C. Posi ons: GET /api/v1/por olio/posi ons

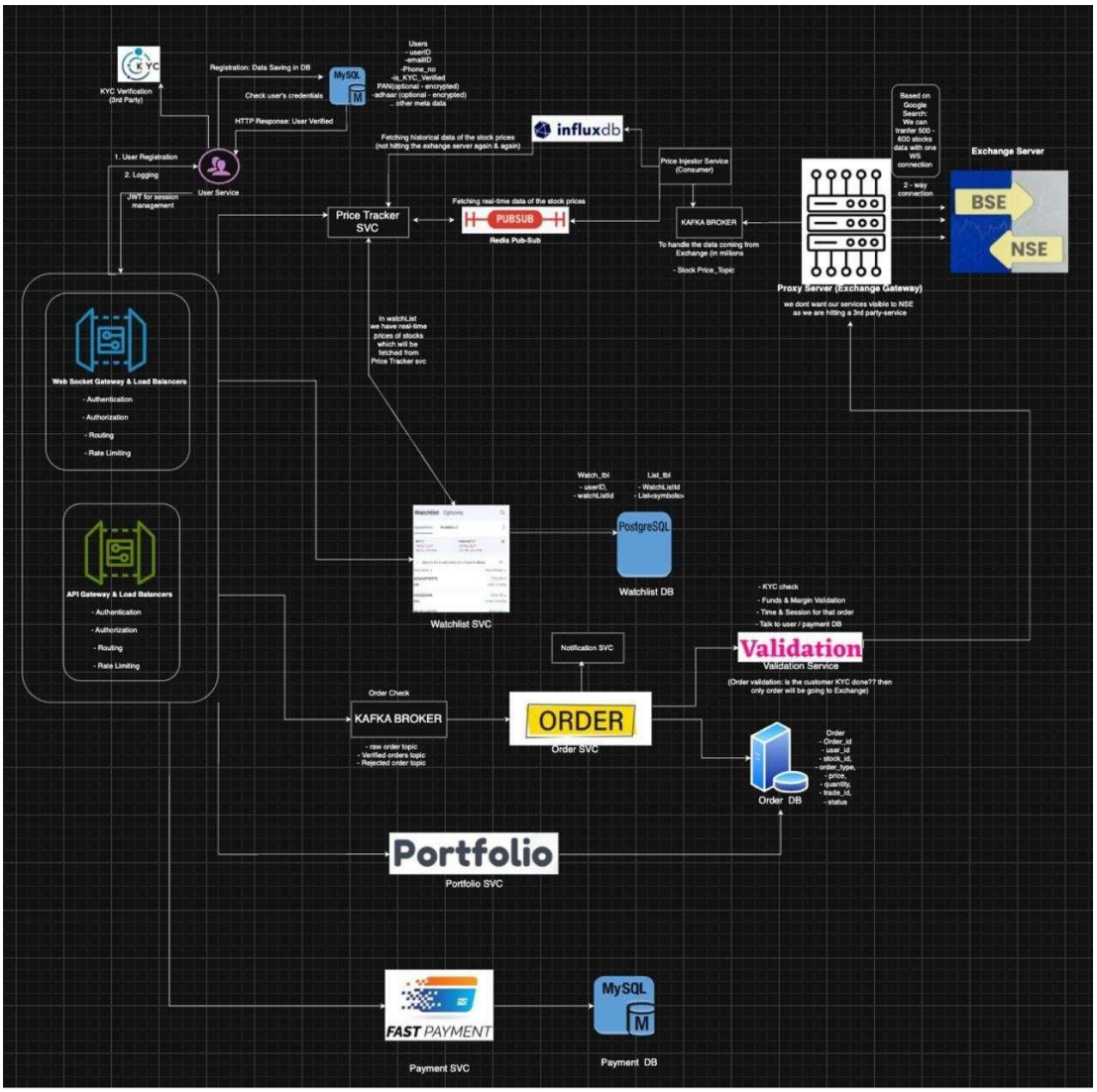
D. Performance: GET /api/v1/por olio/performance

Non-Func onal Requirements

1. System should support 2–3 million daily ac ve users.
2. System should handle 8,000–10,000 stocks.
3. Strong consistency is required for order execu on and balances.
4. Real- me stock updates should be highly available.
5. Order placement latency should be less than 100ms.
6. Market data latency should be less than 50ms.

High Level Design (HLD)

The system consists of client applica ons (web and mobile), an API Gateway, and microservices such as User Service, Market Data Service, Order Management Service, Por olio Service, and Funds Service. Real- me stock updates are delivered using WebSockets. Orders are processed through a low-latency order matching engine. Message queues are used for asynchronous processing of trades and no fica ons. Strong consistency is maintained for financial transac ons, while market data priori zes availability.



Outcome

- Designed a scalable stock trading system with real- me capabili es.
- Understood the importance of consistency in financial systems.
- Implemented structured API design for trading workflows.
- Learned about low-latency architecture and real- me streaming.